Tôi sẽ giải thích chi tiết cách implement authentication với Passport.js, JWT, và social login cho Express.js backend và Next.js frontend.

Kiến trúc tổng quan

Backend (Express.js):

- Passport.js với các strategies: Local (email/password), Google OAuth, GitHub OAuth
- JWT để tạo access token và refresh token
- Routes cho authentication và protected endpoints

Frontend (Next.js):

- Login forms cho email/password
- Social login buttons
- Lưu JWT tokens (httpOnly cookies hoặc memory)
- Axios interceptors để tự động attach token

Backend Implementation

1. Cài đặt dependencies

```
npm install express passport passport-local passport-google-oauth20 passport-github2 passport-jwt jsonwebtoken bcryptjs cookie-parser cors
```

2. Database Models (ví dụ với MongoDB/Mongoose)

```
// models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  email: { type: String, unique: true, sparse: true },
  password: { type: String }, // null nếu social login
  name: String,
  avatar: String,

  // Social login fields
  googleId: { type: String, sparse: true },
  githubId: { type: String, sparse: true },

  provider: { type: String, enum: ['local', 'google', 'github'] },

  refreshTokens: [String], // Store refresh tokens
}, { timestamps: true });

// Hash password trước khi save
userSchema.pre('save', async function(next) {
```

```
  if (!this.isModified('password') || !this.password) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// Method so sánh password
userSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

3. JWT Utilities

```
// utils/jwt.js
const jwt = require('jsonwebtoken');

const ACCESS_TOKEN_SECRET = process.env.ACCESS_TOKEN_SECRET;
const REFRESH_TOKEN_SECRET = process.env.REFRESH_TOKEN_SECRET;

exports.generateAccessToken = (userId) => {
  return jwt.sign(
    { userId },
    ACCESS_TOKEN_SECRET,
    { expiresIn: '15m' } // Short-lived
  );
};

exports.generateRefreshToken = (userId) => {
  return jwt.sign(
    { userId },
    REFRESH_TOKEN_SECRET,
    { expiresIn: '7d' } // Long-lived
  );
};

exports.verifyAccessToken = (token) => {
  return jwt.verify(token, ACCESS_TOKEN_SECRET);
};

exports.verifyRefreshToken = (token) => {
  return jwt.verify(token, REFRESH_TOKEN_SECRET);
};
```

4. Passport Configuration

```
// config/passport.js
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
```

```
const GoogleStrategy = require('passport-google-oauth20').Strategy;
const GithubStrategy = require('passport-github2').Strategy;
const JwtStrategy = require('passport-jwt').Strategy;
const { ExtractJwt } = require('passport-jwt');
const User = require('../models/User');

// Local Strategy (Email/Password)
passport.use(new LocalStrategy(
  {
    usernameField: 'email',
    passwordField: 'password'
  },
  async (email, password, done) => {
    try {
      const user = await User.findOne({ email });

      if (!user) {
        return done(null, false, { message: 'Email không tồn tại' });
      }

      if (!user.password) {
        return done(null, false, { message: 'Tài khoản này dùng social login' });
      }

      const isMatch = await user.comparePassword(password);
      if (!isMatch) {
        return done(null, false, { message: 'Mật khẩu không đúng' });
      }

      return done(null, user);
    } catch (err) {
      return done(err);
    }
  }
));

// Google OAuth Strategy
passport.use(new GoogleStrategy(
  {
    clientID: process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    callbackURL: '/api/auth/google/callback'
  },
  async (accessToken, refreshToken, profile, done) => {
    try {
      // Tìm user với googleId
      let user = await User.findOne({ googleId: profile.id });

      if (!user) {
        // Hoặc tìm theo email nếu đã có account
        user = await User.findOne({ email: profile.emails[0].value });

        if (user) {
          // Link google account với existing user
```

```
          user.googleId = profile.id;
          await user.save();
        } else {
          // Tạo user mới
          user = await User.create({
            googleId: profile.id,
            email: profile.emails[0].value,
            name: profile.displayName,
            avatar: profile.photos[0].value,
            provider: 'google'
          });
        }
      }

      return done(null, user);
    } catch (err) {
      return done(err);
    }
  }
));

// GitHub OAuth Strategy
passport.use(new GithubStrategy(
  {
    clientID: process.env.GITHUB_CLIENT_ID,
    clientSecret: process.env.GITHUB_CLIENT_SECRET,
    callbackURL: '/api/auth/github/callback',
    scope: ['user:email']
  },
  async (accessToken, refreshToken, profile, done) => {
    try {
      let user = await User.findOne({ githubId: profile.id });

      if (!user) {
        const email = profile.emails && profile.emails[0].value;
        user = await User.findOne({ email });

        if (user) {
          user.githubId = profile.id;
          await user.save();
        } else {
          user = await User.create({
            githubId: profile.id,
            email,
            name: profile.displayName || profile.username,
            avatar: profile.photos[0].value,
            provider: 'github'
          });
        }
      }

      return done(null, user);
    } catch (err) {
      return done(err);
```

```
    }
  }
));

// JWT Strategy (cho protected routes)
const jwtOptions = {
  jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
  secretOrKey: process.env.ACCESS_TOKEN_SECRET
};

passport.use(new JwtStrategy(jwtOptions, async (payload, done) => {
  try {
    const user = await User.findById(payload.userId);
    if (!user) return done(null, false);
    return done(null, user);
  } catch (err) {
    return done(err, false);
  }
}));

module.exports = passport;
```

5. Authentication Routes

```
// routes/auth.js
const express = require('express');
const passport = require('passport');
const User = require('../models/User');
const { generateAccessToken, generateRefreshToken, verifyRefreshToken } =
require('../utils/jwt');

const router = express.Router();

// Register với email/password
router.post('/register', async (req, res) => {
  try {
    const { email, password, name } = req.body;

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ message: 'Email đã tồn tại' });
    }

    const user = await User.create({
      email,
      password,
      name,
      provider: 'local'
    });

    const accessToken = generateAccessToken(user._id);
```

```
    const refreshToken = generateRefreshToken(user._id);

    // Lưu refresh token vào DB
    user.refreshTokens.push(refreshToken);
    await user.save();

    // Set httpOnly cookie cho refresh token
    res.cookie('refreshToken', refreshToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 7 * 24 * 60 * 60 * 1000 // 7 days
    });

    res.json({
      accessToken,
      user: {
        id: user._id,
        email: user.email,
        name: user.name,
        avatar: user.avatar
      }
    });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// Login với email/password
router.post('/login', (req, res, next) => {
  passport.authenticate('local', { session: false }, async (err, user, info) => {
    if (err) return next(err);

    if (!user) {
      return res.status(401).json({ message: info.message });
    }

    const accessToken = generateAccessToken(user._id);
    const refreshToken = generateRefreshToken(user._id);

    user.refreshTokens.push(refreshToken);
    await user.save();

    res.cookie('refreshToken', refreshToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 7 * 24 * 60 * 60 * 1000
    });

    res.json({
      accessToken,
      user: {
        id: user._id,
```

```
        email: user.email,
        name: user.name,
        avatar: user.avatar
      }
    });
  })(req, res, next);
});

// Google OAuth
router.get('/google',
  passport.authenticate('google', {
    scope: ['profile', 'email'],
    session: false
  })
);

router.get('/google/callback',
  passport.authenticate('google', { session: false, failureRedirect: '/login' }),
  async (req, res) => {
    const accessToken = generateAccessToken(req.user._id);
    const refreshToken = generateRefreshToken(req.user._id);

    req.user.refreshTokens.push(refreshToken);
    await req.user.save();

    res.cookie('refreshToken', refreshToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 7 * 24 * 60 * 60 * 1000
    });

    // Redirect về frontend với token
    res.redirect(`${process.env.FRONTEND_URL}/auth/callback?
token=${accessToken}`);
  }
);

// GitHub OAuth (tương tự Google)
router.get('/github',
  passport.authenticate('github', { scope: ['user:email'], session: false })
);

router.get('/github/callback',
  passport.authenticate('github', { session: false, failureRedirect: '/login' }),
  async (req, res) => {
    const accessToken = generateAccessToken(req.user._id);
    const refreshToken = generateRefreshToken(req.user._id);

    req.user.refreshTokens.push(refreshToken);
    await req.user.save();

    res.cookie('refreshToken', refreshToken, {
      httpOnly: true,
```

```
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 7 * 24 * 60 * 60 * 1000
    });

    res.redirect(`${process.env.FRONTEND_URL}/auth/callback?
token=${accessToken}`);
  }
);

// Refresh token endpoint
router.post('/refresh', async (req, res) => {
  const refreshToken = req.cookies.refreshToken;

  if (!refreshToken) {
    return res.status(401).json({ message: 'No refresh token' });
  }

  try {
    const payload = verifyRefreshToken(refreshToken);
    const user = await User.findById(payload.userId);

    if (!user || !user.refreshTokens.includes(refreshToken)) {
      return res.status(403).json({ message: 'Invalid refresh token' });
    }

    const newAccessToken = generateAccessToken(user._id);

    res.json({ accessToken: newAccessToken });
  } catch (err) {
    res.status(403).json({ message: 'Invalid refresh token' });
  }
});

// Logout
router.post('/logout', async (req, res) => {
  const refreshToken = req.cookies.refreshToken;

  if (refreshToken) {
    try {
      const payload = verifyRefreshToken(refreshToken);
      const user = await User.findById(payload.userId);

      if (user) {
        user.refreshTokens = user.refreshTokens.filter(t => t !== refreshToken);
        await user.save();
      }
    } catch (err) {
      // Token invalid, ignore
    }
  }

  res.clearCookie('refreshToken');
  res.json({ message: 'Logged out' });
```

```
});

// Get current user
router.get('/me',
  passport.authenticate('jwt', { session: false }),
  (req, res) => {
    res.json({
      user: {
        id: req.user._id,
        email: req.user.email,
        name: req.user.name,
        avatar: req.user.avatar
      }
    });
  }
);

module.exports = router;
```

6. Main Express App

```
// app.js
const express = require('express');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const passport = require('./config/passport');

const app = express();

app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true
}));

app.use(express.json());
app.use(cookieParser());
app.use(passport.initialize());

// Routes
app.use('/api/auth', require('./routes/auth'));

// Protected route example
app.get('/api/protected',
  passport.authenticate('jwt', { session: false }),
  (req, res) => {
    res.json({ message: 'This is protected data', user: req.user });
  }
);

module.exports = app;
```

Frontend (Next.js) Implementation

1. Cài đặt dependencies

```
npm install axios js-cookie
```

2. API Client với Axios

```
// lib/api.js
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:5000/api',
  withCredentials: true // Important cho cookies
});

let accessToken = null;

export const setAccessToken = (token) => {
  accessToken = token;
};

export const getAccessToken = () => accessToken;

// Interceptor tự động attach access token
api.interceptors.request.use(
  (config) => {
    if (accessToken) {
      config.headers.Authorization = `Bearer ${accessToken}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Interceptor tự động refresh token khi expired
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;

    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;

      try {
        const { data } = await axios.post(
          `${process.env.NEXT_PUBLIC_API_URL}/auth/refresh`,
          {},
          { withCredentials: true }
        );
```

```
        setAccessToken(data.accessToken);
        originalRequest.headers.Authorization = `Bearer ${data.accessToken}`;

        return api(originalRequest);
      } catch (refreshError) {
        setAccessToken(null);
        window.location.href = '/login';
        return Promise.reject(refreshError);
      }
    }

    return Promise.reject(error);
  }
);

export default api;
```

3. Auth Context

```
// context/AuthContext.js
'use client';
import { createContext, useContext, useState, useEffect } from 'react';
import api, { setAccessToken, getAccessToken } from '@/lib/api';

const AuthContext = createContext();

export const useAuth = () => useContext(AuthContext);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    checkAuth();
  }, []);

  const checkAuth = async () => {
    try {
      // Try refresh token
      const { data } = await api.post('/auth/refresh');
      setAccessToken(data.accessToken);

      // Get user info
      const userRes = await api.get('/auth/me');
      setUser(userRes.data.user);
    } catch (err) {
      setUser(null);
      setAccessToken(null);
    } finally {
      setLoading(false);
```

```
    }
  };

  const login = async (email, password) => {
    const { data } = await api.post('/auth/login', { email, password });
    setAccessToken(data.accessToken);
    setUser(data.user);
    return data;
  };

  const register = async (email, password, name) => {
    const { data } = await api.post('/auth/register', { email, password, name });
    setAccessToken(data.accessToken);
    setUser(data.user);
    return data;
  };

  const logout = async () => {
    await api.post('/auth/logout');
    setAccessToken(null);
    setUser(null);
  };

  const loginWithGoogle = () => {
    window.location.href = `${process.env.NEXT_PUBLIC_API_URL}/auth/google`;
  };

  const loginWithGithub = () => {
    window.location.href = `${process.env.NEXT_PUBLIC_API_URL}/auth/github`;
  };

  return (
    <AuthContext.Provider value={{
      user,
      loading,
      login,
      register,
      logout,
      loginWithGoogle,
      loginWithGithub
    }}>
      {children}
    </AuthContext.Provider>
  );
};
```

4. Login Page

```
// app/login/page.js
'use client';
import { useState } from 'react';
```

```
import { useAuth } from '@/context/AuthContext';
import { useRouter } from 'next/navigation';

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const { login, loginWithGoogle, loginWithGithub } = useAuth();
  const router = useRouter();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    try {
      await login(email, password);
      router.push('/dashboard');
    } catch (err) {
      setError(err.response?.data?.message || 'Login failed');
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center">
      <div className="max-w-md w-full space-y-8 p-8">
        <h2 className="text-3xl font-bold text-center">Login</h2>

        {error && (
          <div className="bg-red-100 border border-red-400 text-red-700 px-4 py-3
rounded">
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-6">
          <div>
            <label className="block text-sm font-medium">Email</label>
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="mt-1 block w-full px-3 py-2 border border-gray-300
rounded-md"
              required
            />
          </div>

          <div>
            <label className="block text-sm font-medium">Password</label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="mt-1 block w-full px-3 py-2 border border-gray-300
```

```
rounded-md"
              required
            />
          </div>

          <button
            type="submit"
            className="w-full bg-blue-600 text-white py-2 px-4 rounded-md
hover:bg-blue-700"
          >
            Login
          </button>
        </form>

        <div className="relative">
          <div className="absolute inset-0 flex items-center">
            <div className="w-full border-t border-gray-300"></div>
          </div>
          <div className="relative flex justify-center text-sm">
            <span className="px-2 bg-white text-gray-500">Or continue with</span>
          </div>
        </div>

        <div className="grid grid-cols-2 gap-3">
          <button
            onClick={loginWithGoogle}
            className="w-full border border-gray-300 py-2 px-4 rounded-md
hover:bg-gray-50"
          >
            Google
          </button>

          <button
            onClick={loginWithGithub}
            className="w-full border border-gray-300 py-2 px-4 rounded-md
hover:bg-gray-50"
          >
            GitHub
          </button>
        </div>
      </div>
    </div>
  );
}
```

5. OAuth Callback Handler

```
// app/auth/callback/page.js
'use client';
import { useEffect } from 'react';
import { useRouter, useSearchParams } from 'next/navigation';
```

```
import { setAccessToken } from '@/lib/api';
import api from '@/lib/api';
import { useAuth } from '@/context/AuthContext';

export default function AuthCallback() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const { checkAuth } = useAuth();

  useEffect(() => {
    const token = searchParams.get('token');

    if (token) {
      setAccessToken(token);

      // Fetch user data
      api.get('/auth/me')
        .then(() => {
          router.push('/dashboard');
        })
        .catch(() => {
          router.push('/login');
        });
    } else {
      router.push('/login');
    }
  }, [searchParams, router]);

  return (
    <div className="min-h-screen flex items-center justify-center">
      <div>Loading...</div>
    </div>
  );
}
```

6. Protected Route Component

```
// components/ProtectedRoute.js
'use client';
import { useAuth } from '@/context/AuthContext';
import { useRouter } from 'next/navigation';
import { useEffect } from 'react';

export default function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!loading && !user) {
      router.push('/login');
    }
```

```
  }, [user, loading, router]);

  if (loading) {
    return <div>Loading...</div>;
  }

  if (!user) {
    return null;
  }

  return children;
}
```

7. Environment Variables

.env.local (Frontend)

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
```

.env (Backend)

```
PORT=5000
FRONTEND_URL=http://localhost:3000
ACCESS_TOKEN_SECRET=your-access-token-secret
REFRESH_TOKEN_SECRET=your-refresh-token-secret

GOOGLE_CLIENT_ID=your-google-client-id
GOOGLE_CLIENT_SECRET=your-google-client-secret

GITHUB_CLIENT_ID=your-github-client-id
GITHUB_CLIENT_SECRET=your-github-client-secret

MONGODB_URI=mongodb://localhost:27017/your-db
```

Flow tổng quan

1. **Local Login**: User nhập email/password → Backend verify → Trả về access token + refresh token (httpOnly cookie)
2. **Social Login**: User click Google/GitHub → Redirect sang OAuth provider → Callback về backend → Backend tạo/link user → Redirect về frontend với token
3. **Token Management**:
   - Access token (short-lived, 15 phút) được lưu trong memory
   - Refresh token (long-lived, 7 ngày) được lưu trong httpOnly cookie
   - Khi access token hết hạn, axios interceptor tự động call /refresh endpoint

4. **Protected Routes**: Axios tự động attach Bearer token vào headers, backend verify bằng JWT strategy

Bạn có cần giải thích thêm phần nào không?